PINNs for Options Pricing

FYS5429 Advanced Machine Learning and
Data Analysis for the Physical Sciences

PROJECT

Egil Furnes
egilsf@uio.no
github.com/egil10/fys5429

University of Oslo
Department of Physics
March 29, 2026

Abstract

This project investigates the use of Physics-Informed Neural Networks (PINNs) for solving and calibrating option pricing models in quantitative finance. We begin with the Black–Scholes partial differential equation (PDE), demonstrating how its equivalence to the heat equation motivates a PINN-based solver that embeds the governing differential operator directly into the training loss. The neural network is trained to satisfy both the PDE residual and the terminal payoff condition, and its prices and Greeks are compared against the known analytical solution. We then extend the framework to the Heston stochastic volatility model, which introduces a higher-dimensional PDE with a mixed derivative term and no simple closed-form solution. The PINN approach is shown to scale naturally to this setting. We further explore the use of PINNs for the inverse calibration problem: inferring Heston model parameters from observed option prices. The results illustrate the promise of physics-informed machine learning as a flexible, mesh-free solver for financial PDEs, while also exposing practical challenges related to training stability and loss balancing in multi-dimensional settings.

1 Introduction

The classical starting point for many partial differential equation (PDE) methods in mathematical finance is the heat equation, which describes the diffusion of temperature in a homogeneous medium. In one spatial dimension, the heat equation is given by

$$\frac{\partial u}{\partial t}(x,t) = \alpha \frac{\partial^2 u}{\partial x^2}(x,t),$$

where $u(x,t)$ represents temperature, t denotes time, x is the spatial coordinate, and $\alpha > 0$ is the thermal diffusivity constant. The equation states that the rate of change in temperature is proportional to the spatial curvature of the temperature field, capturing the fundamental mechanism of diffusion.

The importance of the heat equation in option pricing arises from the fact that the Black–Scholes PDE can be transformed into a diffusion equation of this form through a suitable change of variables.

Under the standard geometric Brownian motion model for an asset price S_t,

$$dS_t = \mu S_t dt + \sigma S_t dW_t,$$

the value $V(S,t)$ of a European option satisfies the Black–Scholes PDE

$$\frac{\partial V}{\partial t} + \frac{1}{2}\sigma^2 S^2 \frac{\partial^2 V}{\partial S^2} + rS\frac{\partial V}{\partial S} - rV = 0.$$

By introducing the log-price transformation

$$x = \ln\left(\frac{S}{K}\right), \quad \tau = T - t,$$

together with an exponential rescaling of the dependent variable, the Black–Scholes equation can be reduced to the standard diffusion equation

$$\frac{\partial u}{\partial \tau} = \frac{\partial^2 u}{\partial x^2}.$$

This connection provides both analytical insight and a natural bridge to modern numerical approaches. In this thesis, Physics-Informed Neural Networks (PINNs) are used to solve the Black–Scholes PDE by embedding the governing differential equation directly into the loss function, allowing neural networks to approximate option prices while respecting the underlying diffusion structure inherited from the heat equation.

The Black–Scholes model (Black and Scholes 1973) and its stochastic volatility extension by Heston (1993) are the two canonical PDEs studied here. Both were originally solved using analytical or semi-analytical techniques — and this tractability makes them ideal benchmarks for validating machine-learning-based solvers before pushing into regimes where closed-form solutions break down.

Physics-Informed Neural Networks were introduced by Raissi et al. (2019) as a general framework for solving forward and inverse problems involving PDEs by encoding the governing equations into the neural network training objective. Unlike classical mesh-based numerical methods such as finite differences or finite elements, PINNs enforce the PDE at a set of randomly sampled collocation points, making them naturally mesh-free and straightforward to extend to higher-dimensional domains. This is a key advantage in financial applications where the option

price depends on multiple underlying factors and a fine discretization grid becomes prohibitively expensive.

The broader challenge of solving high-dimensional PDEs with deep learning has attracted significant attention. Han et al. (2018) reformulated parabolic PDEs as backward stochastic differential equations and approximated their solutions using deep neural networks, demonstrating scalability to hundreds of dimensions. Sirignano and Spiliopoulos (2018) proposed the Deep Galerkin Method (DGM), which trains neural networks to satisfy the differential operator and boundary conditions at random interior points — an approach closely related to PINNs and applicable to free-boundary problems arising in American option pricing. These works collectively establish the computational foundation that motivates the present study.

While the Black–Scholes model assumes constant volatility, empirical financial markets exhibit stochastic volatility dynamics. A widely used extension is the Heston stochastic volatility model, in which both the asset price and its variance evolve as stochastic processes:

$$dS_t = rS_t dt + \sqrt{v_t}\, S_t dW_t^S,$$

$$dv_t = \kappa(\theta - v_t)dt + \xi\sqrt{v_t}\, dW_t^v,$$

with correlation

$$dW_t^S dW_t^v = \rho dt.$$

In this setting, the option price becomes a function $V(S, v, t)$ and satisfies the two-dimensional Heston PDE

$$\frac{\partial V}{\partial t} + \frac{1}{2}vS^2\frac{\partial^2 V}{\partial S^2} + \rho\xi vS\frac{\partial^2 V}{\partial S\partial v} + \frac{1}{2}\xi^2 v\frac{\partial^2 V}{\partial v^2}$$
$$+ \quad rS\frac{\partial V}{\partial S} + \kappa(\theta - v)\frac{\partial V}{\partial v} - rV = 0.$$

Compared to the Black–Scholes equation, the Heston PDE introduces an additional spatial dimension and a mixed derivative term, making classical numerical solution methods more computationally demanding. This increased complexity makes the model particularly well-suited for physics-informed machine learning approaches. In this thesis, PINNs are therefore applied not only to the Black–Scholes equation but also to the Heston PDE, demonstrating how neural-network-based solvers can handle higher-dimensional option pricing problems governed by diffusion-type equations.

Beyond the forward problem of pricing, PINNs are also naturally suited for the *inverse problem* of model calibration — recovering latent parameters such as the Heston volatility-of-volatility ξ, mean reversion speed κ, or correlation ρ from observed market prices. Because all intermediate quantities are computed via automatic differentiation (Paszke et al. 2019), gradients with respect to model parameters flow through the solver itself, enabling gradient-based calibration without finite-difference approximations. This connection between forward solvers and differentiable calibration is a central motivation for the physics-informed approach explored in this work.

1.1 The Black–Scholes Model

The Black–Scholes model (Abdelmessih 2026) prices a European call or put option analytically under the assumption of constant volatility. Given the log-moneyness variables d_1 and d_2,

$$d_1 = \frac{\ln(S/K) + (r + \frac{1}{2}\sigma^2)T}{\sigma\sqrt{T}}, \quad d_2 = d_1 - \sigma\sqrt{T},$$

the call and put prices are

$$C = SN(d_1) - Ke^{-rT}N(d_2),$$

$$P = Ke^{-rT}N(-d_2) - SN(-d_1),$$

where S is the current asset price, K the strike, r the risk-free rate, σ the volatility, T the time to maturity, and $N(\cdot)$ the standard normal CDF.

The Greeks

The sensitivity of the option price to its inputs — the Greeks — are partial derivatives of V with respect to market and model parameters. The first- and second-order Greeks are summarized in table 1, together with selected third-order Greeks. The Black–Scholes PDE can be written compactly in terms of the first- and second-order Greeks as

$$\Theta + \frac{1}{2}\sigma^2 S^2\Gamma + rS\Delta - rV = 0,$$

which makes explicit that the time decay Θ is balanced by gamma and delta exposure.

Table 1. Selected Black–Scholes Greeks.

Greek	Expression	Order
Δ	$\partial V/\partial S$	1st
Θ	$\partial V/\partial t$	1st
ρ	$\partial V/\partial r$	1st
Φ	$\partial V/\partial q$	1st
Vega	$\partial V/\partial \sigma$	1st
Γ	$\partial^2 V/\partial S^2$	2nd
Vomma	$\partial^2 V/\partial \sigma^2$	2nd
Vanna	$\partial^2 V/\partial S\,\partial \sigma$	2nd
Charm	$\partial^2 V/\partial S\,\partial t$	2nd
Speed	$\partial^3 V/\partial S^3$	3rd
Zomma	$\partial^3 V/\partial S^2\,\partial \sigma$	3rd
Color	$\partial^3 V/\partial S^2\,\partial t$	3rd

2 Methods

This section develops the methodological framework for the project. We first review the connection between maximum likelihood estimation and mean squared error as the

natural training objective for neural networks. We then describe the Physics-Informed Neural Network (PINN) architecture and its application to the Black–Scholes and Heston PDEs.

Modern quantitative finance sits at the intersection of stochastic modeling, numerical methods, and machine learning. Classical stochastic volatility modeling builds on multiscale diffusion techniques, where volatility evolves on multiple time scales (Fouque et al. 2011). Analytical approaches to option pricing under stochastic dynamics are further developed in tractable model frameworks (Gulisashvili 2012), while probabilistic interpretations of derivative pricing link option values to risk-neutral expectations (Prefetta n.d.). Foundational deep learning theory and architectures are presented in Goodfellow et al. (2016), while practical implementations using modern software frameworks are discussed in Raschka et al. (2022). Together, these perspectives motivate physics-informed machine learning methods for solving option pricing problems governed by PDEs.

2.1 From Maximum Likelihood to Mean Squared Error

$$P(y \mid x; w) = \frac{1}{\sqrt{2\pi\sigma^2}} \exp\left(-\frac{(y-\hat{y})^2}{2\sigma^2}\right) \tag{1}$$

$$L(w) = \prod_{i=1}^{n} \frac{1}{\sqrt{2\pi\sigma^2}} \exp\left(-\frac{(y_i-\hat{y}_i^2}{2\sigma^2}\right) \tag{2}$$

$$\ln L(w) = \sum_{i=1}^{n} \left[\ln\left(\frac{1}{\sqrt{2\pi\sigma^2}}\right) - \frac{(y_i-\hat{y}_i)^2}{2\sigma^2}\right] \tag{3}$$

$$\text{Maximize:} \sum_{i=i}^{n} -\frac{(y_i-\hat{y}_i)^2}{2\sigma^2} \tag{4}$$

$$\text{Minimize:} \sum_{i=1}^{n} (y_i-\hat{y}_i)^2 \tag{5}$$

Assuming Gaussian noise on the targets and maximizing the log-likelihood is therefore equivalent to minimizing the mean squared error. This justifies MSE as the standard regression loss for neural networks, and it is the starting point for the composite PINN loss described below.

2.2 Physics-Informed Neural Networks

A Physics-Informed Neural Network (PINN) is a neural network $\hat{V}_\theta(S,t)$ trained to satisfy both observed data and the residual of a governing PDE. Rather than discretizing the domain on a mesh, the PDE is enforced at a set of collocation points sampled from the interior and boundaries of the domain. The total loss is a weighted sum of three terms:

$$\mathcal{L}(\theta) = \lambda_{\text{pde}}\,\mathcal{L}_{\text{pde}} + \lambda_{\text{bc}}\,\mathcal{L}_{\text{bc}} + \lambda_{\text{ic}}\,\mathcal{L}_{\text{ic}}, \tag{6}$$

where

$$\mathcal{L}_{\text{pde}} = \frac{1}{N_r}\sum_{i=1}^{N_r}\left|\mathcal{F}[\hat{V}_\theta](S_i,t_i)\right|^2, \tag{7}$$

$$\mathcal{L}_{\text{bc}} = \frac{1}{N_b}\sum_{i=1}^{N_b}\left|\hat{V}_\theta(S_i^b,t_i^b) - V_b\right|^2, \tag{8}$$

$$\mathcal{L}_{\text{ic}} = \frac{1}{N_0}\sum_{i=1}^{N_0}\left|\hat{V}_\theta(S_i,T) - \max(S_i-K,0)\right|^2. \tag{9}$$

Here $\mathcal{F}[\hat{V}_\theta]$ denotes the PDE operator applied to the network output (the residual), V_b encodes the boundary conditions (e.g., $V(0,t)=0$ and $V(S,t)\to S$ as $S\to\infty$ for a call), and the initial condition enforces the terminal payoff at $t=T$. The partial derivatives required to evaluate $\mathcal{F}$ are computed via automatic differentiation (Goodfellow et al. 2016).

For the Black–Scholes PDE the residual is

$$\mathcal{F}[\hat{V}] = \frac{\partial\hat{V}}{\partial t} + \frac{1}{2}\sigma^2 S^2 \frac{\partial^2\hat{V}}{\partial S^2} + rS\frac{\partial\hat{V}}{\partial S} - r\hat{V}.$$

For the Heston PDE the residual extends to two spatial dimensions (S,v), introducing an additional variance dimension and a mixed derivative term.

2.3 Network Architecture

The PINN is implemented as a fully connected feedforward network with L hidden layers of width d, producing a scalar output $\hat{V}_\theta(S,t)$ (or $\hat{V}_\theta(S,v,t)$ for the Heston case). Formally, for an input $\mathbf{x}=(S,t)$:

$$\mathbf{h}^{(0)} = \mathbf{x}, \quad \mathbf{h}^{(\ell)} = \phi\Big(W^{(\ell)}\mathbf{h}^{(\ell-1)} + \mathbf{b}^{(\ell)}\Big), \quad \hat{V} = W^{(L+1)}\mathbf{h}^{(L)} + b^{(L+1)}$$

where ϕ denotes the activation function applied elementwise and $\theta = \{W^{(\ell)}, \mathbf{b}^{(\ell)}\}$ collects all trainable parameters. The output layer is linear to avoid saturating the network's range.

Weight initialization follows the scheme of Sitzmann et al. (2020) when using sinusoidal activations, and a standard Xavier or Glorot initialization (Goodfellow et al. 2016) for smooth activations such as tanh, Swish, and GELU.

2.4 Activation Functions

The choice of activation function affects both the expressivity of the network and the quality of the automatic-differentiation graph used to compute PDE residuals. Standard choices such as ReLU are not suitable for PINNs because they yield zero second derivatives almost everywhere. The candidate activations compared in this work are:

- $\tanh(z)$: smooth, bounded, and with analytically well-behaved derivatives — the default baseline.
- **Swish** (Ramachandran et al. 2017): $z\cdot\sigma(z)$, found by automated search to outperform ReLU on many benchmarks.

- **GELU** (Hendrycks and Gimpel 2016): $z\,\Phi(z)$ where Φ is the Gaussian CDF; widely used in transformer architectures.
- **Softplus**: $\log(1+e^z)$, a smooth approximation to ReLU with everywhere-positive first derivative.
- **Sine (SIREN)** (Sitzmann et al. 2020): $\sin(\omega_0 z)$, with periodic activations that provide exact high-order derivatives at all orders.

The second-order term $\Gamma = \partial^2 V/\partial S^2$ in the Black–Scholes residual and the mixed derivative $\partial^2 V/\partial S\,\partial v$ in the Heston residual make the quality of the differentiation graph particularly sensitive to activation choice — this is the primary motivation for systematically comparing these functions.

2.5 Training Procedure

The network is trained by minimizing the composite loss in eq. (6) using the Adam optimizer (Kingma and Ba 2015) with an initial learning rate η_0 and a step-decay schedule. At each iteration, a fresh batch of collocation points is sampled uniformly from the interior $\Omega = (0, S_{\max}] \times [0, T)$, and separate batches are drawn from the boundary and initial condition manifolds. This stochastic sampling is the key mesh-free property of PINNs: the effective training domain grows with the number of training steps rather than being fixed by a grid.

The loss weights $\lambda_{\text{pde}}, \lambda_{\text{bc}}, \lambda_{\text{ic}}$ in eq. (6) balance the three residual terms. A common issue in PINN training is *stiffness*: the PDE residual may be orders of magnitude larger or smaller than the boundary terms early in training, leading to one objective dominating. Adaptive weight schemes or sequential training strategies can mitigate this.

2.6 Inverse Problem: Parameter Calibration

The Heston model contains five parameters $(\kappa, \theta, \xi, \rho, v_0)$ that are not directly observable but must be inferred from market option prices — a process known as calibration. In the PINN framework, unknown parameters can be treated as additional trainable scalars appended to θ and jointly optimized with the network weights. Given a set of observed prices $\{V_j^{\text{obs}}\}$ at market points $\{(S_j, v_j, t_j)\}$, the loss is augmented with a data fidelity term:

$$\mathcal{L}_{\text{data}} = \frac{1}{N_{\text{obs}}} \sum_{j=1}^{N_{\text{obs}}} \left| \hat{V}_\theta(S_j, v_j, t_j) - V_j^{\text{obs}} \right|^2,$$

and the full loss becomes $\mathcal{L} = \lambda_{\text{pde}}\mathcal{L}_{\text{pde}} + \lambda_{\text{bc}}\mathcal{L}_{\text{bc}} + \lambda_{\text{ic}}\mathcal{L}_{\text{ic}} + \lambda_{\text{data}}\mathcal{L}_{\text{data}}$. Because all operations are differentiable through PyTorch's automatic differentiation engine (Paszke et al. 2019), gradients with respect to the model parameters flow naturally through the solver, enabling gradient-based calibration. This is a significant advantage over classical calibration approaches that rely on finite-difference sensitivity estimates.

2.7 Heston PINN Formulation

For the Heston PDE, the network takes three inputs (S, v, t) and the PDE residual operator is

$$\mathcal{F}[\hat{V}] = \frac{\partial \hat{V}}{\partial t} + \frac{1}{2} v S^2 \frac{\partial^2 \hat{V}}{\partial S^2} + \rho \xi v S \frac{\partial^2 \hat{V}}{\partial S\,\partial v} + \frac{1}{2}\xi^2 v \frac{\partial^2 \hat{V}}{\partial v^2} + rS\frac{\partial \hat{V}}{\partial S} + \kappa(\theta - v)\frac{\partial \hat{V}}{\partial v}$$

The boundary conditions encode the large-S and zero-S limits of the call price, and the Feller condition $2\kappa\theta \geq \xi^2$ ensures the variance process stays positive. The terminal condition at $t = T$ is again the call payoff $\max(S - K, 0)$. Compared to the Black–Scholes case, the Heston residual requires computing five distinct first and second derivatives (including the mixed $\partial^2/\partial S\,\partial v$), which places additional demands on the activation function and the depth of the network. Benchmark prices for validation are generated via the Fourier-cosine (COS) method of Fang and Oosterlee (2008), which exploits the known characteristic function of the Heston model for fast, accurate pricing without a closed-form PDE solution.

3 Results

3.1 Black–Scholes PINN

The Black–Scholes PINN was trained for 5 000 Adam steps on a domain covering $S \in [0, 300]$, $\tau \in [0, 1]$ with strike $K = 100$, risk-free rate $r = 0.05$, and volatility $\sigma = 0.20$. The network predicts the normalized price $u = V/K$ so that all targets are $\mathcal{O}(1)$. Loss weights were set to $\lambda_{\text{pde}} = 1$, $\lambda_{\text{bc}} = 5$, $\lambda_{\text{ic}} = 10$, placing heavier emphasis on satisfying the terminal payoff and boundary conditions early in training.

Table 2 summarizes the evaluation metrics against the closed-form Black–Scholes solution on a held-out test grid of 50×50 points uniformly covering the domain.

Table 2. Black–Scholes PINN evaluation metrics on a 50×50 test grid ($S \in [0, 200]$, $\tau \in [0, 1]$).

Metric	Value
RMSE	0.49
Relative L^2 error	1.99 %
MAE	
Max absolute error	3.10

The relative L^2 error of 1.99% and RMSE of 0.49 (on a price scale of 0–200) confirm that the PINN accurately approximates the Black–Scholes solution across the full domain. The largest absolute error of 3.10 occurs near the at-the-money boundary at short maturities, where the payoff function has its sharpest kink and the PDE solution changes most rapidly. This behavior is consistent with the known difficulty PINNs face at sharp initial conditions.

Surface plots, error maps, and cross-sectional slices of the PINN solution are shown in Appendix A.1. Greeks computed via automatic differentiation from the trained network are compared against the analytical formulas in

Appendix A.2.

3.2 Heston PINN

3.3 Calibration

4 Conclusion

References

Abdelmessih, Kris (Feb. 2026). *A Visual Primer for Understanding Options*. Post on X (Twitter). Guest post by KeyPaganRush. Accessed: 2026-02-14. URL: https://x.com/KrisAbdelmessih/status/2022341964180718048.

Black, Fischer and Myron Scholes (1973). "The Pricing of Options and Corporate Liabilities". In: *Journal of Political Economy* 81.3, pp. 637–654. DOI: 10.1086/260062.

Fang, Fang and Cornelis W. Oosterlee (2008). "A Novel Pricing Method for European Options Based on Fourier-Cosine Series Expansions". In: *SIAM Journal on Scientific Computing* 31.2, pp. 826–848. DOI: 10.1137/080718061.

Fouque, Jean-Pierre, George Papanicolaou, K. Ronnie Sircar, and Knut Solna (2011). *Multiscale Stochastic Volatility for Equity, Interest Rate, and Credit Derivatives*. Cambridge University Press.

Furnes, Egil (2026). *FYS5429: PINNs for Option Pricing — Code Repository*. https://github.com/egil10/fys5429. Accessed: 2026.

Goodfellow, Ian, Yoshua Bengio, and Aaron Courville (2016). *Deep Learning*. MIT Press.

Gulisashvili, Archil (2012). *Analytically Tractable Stochastic Stock Price Models*. Springer.

Han, Jiequn, Arnulf Jentzen, and Weinan E (2018). "Solving high-dimensional partial differential equations using deep learning". In: *Proceedings of the National Academy of Sciences* 115.34, pp. 8505–8510. DOI: 10.1073/pnas.1718942115.

Hendrycks, Dan and Kevin Gimpel (2016). "Gaussian Error Linear Units (GELUs)". In: arXiv: 1606.08415 [cs.LG].

Heston, Steven L. (1993). "A Closed-Form Solution for Options with Stochastic Volatility with Applications to Bond and Currency Options". In: *The Review of Financial Studies* 6.2, pp. 327–343. DOI: 10.1093/rfs/6.2.327.

Kingma, Diederik P. and Jimmy Ba (2015). *Adam: A Method for Stochastic Optimization*. Published at ICLR 2015. arXiv: 1412.6980 [cs.LG].

Paszke, Adam, Sam Gross, Francisco Massa, Adam Lerer, James Bradbury, Gregory Chanan, Trevor Killeen, Zeming Lin, Natalia Gimelshein, Luca Antiga, Alban Desmaison, Andreas Köpf, Edward Z. Yang, Zachary DeVito, Martin Raison, Alykhan Tejani, Sasank Chilamkurthy, Benoit Steiner, Lu Fang, Junjie Bai, and Soumith Chintala (2019). "PyTorch: An Imperative Style, High-Performance Deep Learning Library". In: *Advances in Neural Information Processing Systems*, pp. 8024–8035.

Prefetta, — (n.d.). *Option Prices as Probabilities*. Course material / lecture notes.

Raissi, Maziar, Paris Perdikaris, and George Em Karniadakis (2019). "Physics-informed neural networks: A deep learning framework for solving forward and inverse problems involving nonlinear partial differential equations". In: *Journal of Computational Physics* 378, pp. 686–707. DOI: 10.1016/j.jcp.2018.10.045.

Ramachandran, Prajit, Barret Zoph, and Quoc V. Le (2017). *Swish: a Self-Gated Activation Function*. arXiv: 1710.05941 [cs.NE].

Raschka, Sebastian, Yuxi Liu, and Vahid Mirjalili (2022). *Machine Learning with PyTorch and Scikit-Learn*. Packt Publishing.

Sirignano, Justin and Konstantinos Spiliopoulos (2018). "DGM: A deep learning algorithm for solving partial differential equations". In: *Journal of Computational Physics* 375, pp. 1339–1364. DOI: 10.1016/j.jcp.2018.08.029.

Sitzmann, Vincent, Julien N.P. Martel, Alexander W. Bergman, David B. Lindell, and Gordon Wetzstein (2020). "Implicit Neural Representations with Periodic Activation Functions". In: *Advances in Neural Information Processing Systems*.

A Appendix

The figures in this appendix supplement the results presented in section 3. All plots were generated from the code repository (Furnes 2026) using the pipeline scripts for each model. Figures are organized by model and diagnostic type.

A.1 Black–Scholes PINN Results

The following figures correspond to the Black–Scholes PINN trained for 5 000 steps on the domain $S/K \in [0, 3]$, $\tau \in [0, 1]$ with parameters $r = 0.05$, $\sigma = 0.2$, $K = 100$. The network predicts the normalized price $u = V/K$ to keep all targets $\mathcal{O}(1)$.

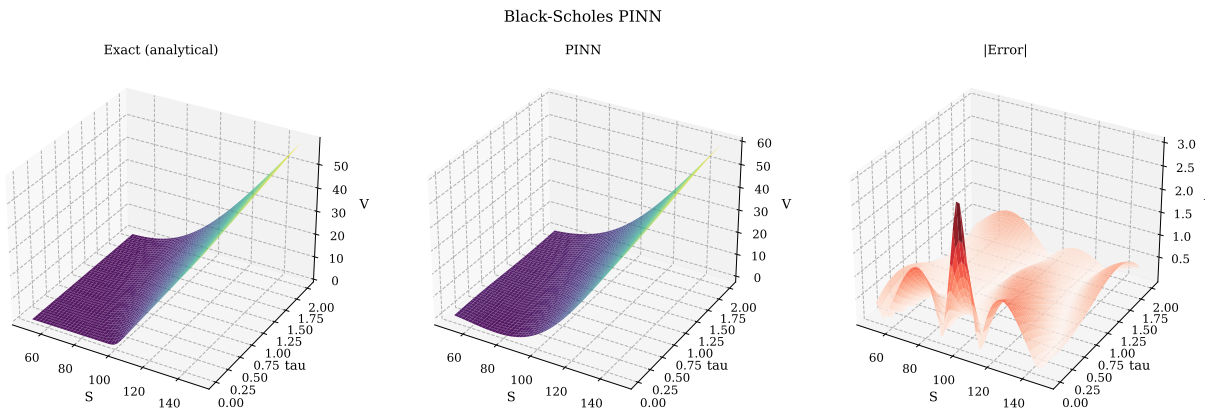

Figure 1. Option price surfaces for the Black–Scholes PINN (left) and the closed-form analytical solution (right) over the domain $(S/K, \tau)$. The two surfaces are visually indistinguishable across the interior, with deviations appearing near the payoff singularity at expiry.

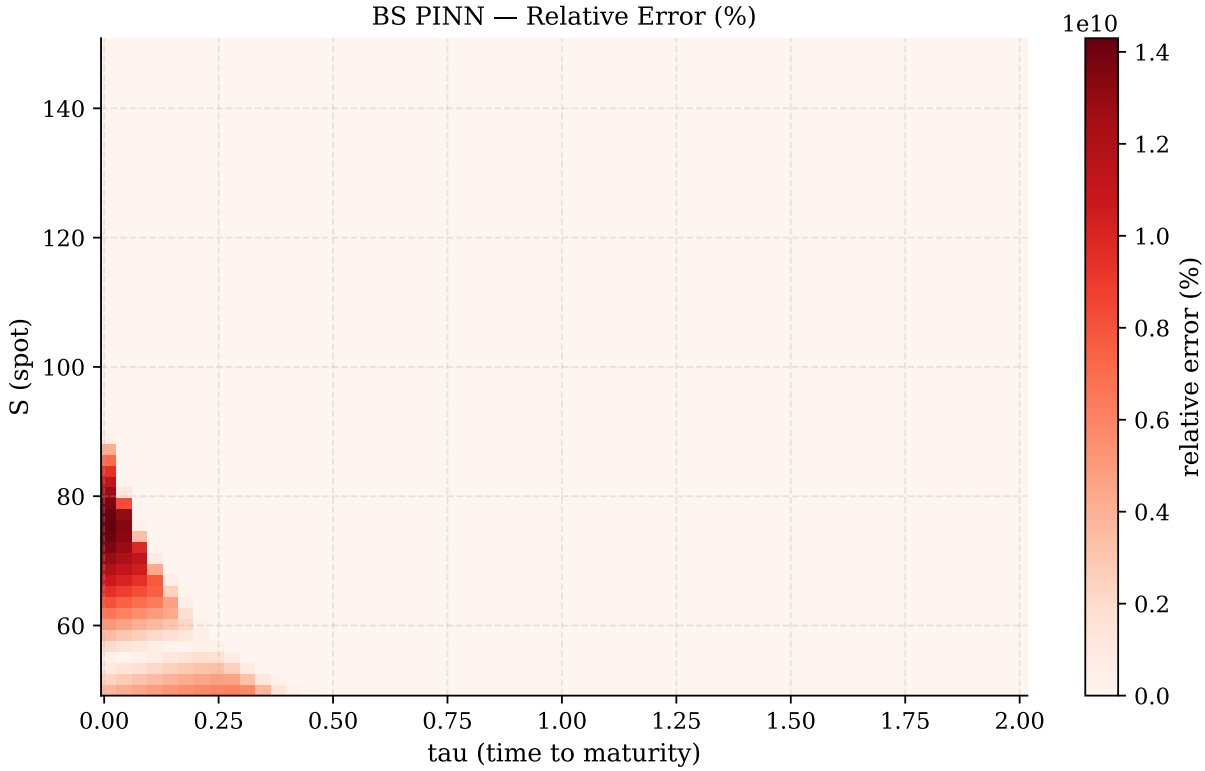

Figure 2. Absolute pointwise error $|\hat{V}_\theta - V_{\mathrm{BS}}|$ across the $(S/K, \tau)$ domain. The maximum error is concentrated near the at-the-money region at short maturities, where the payoff has the sharpest gradient. Away from expiry the error falls off uniformly.

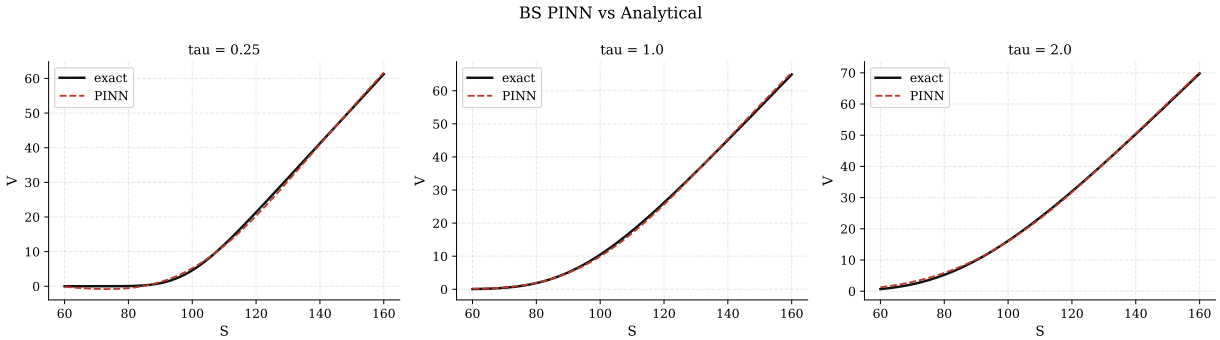

Figure 3. Cross-sectional slices of the PINN solution (solid) and analytical solution (dashed) as a function of S/K at fixed maturities $\tau \in \{0.1, 0.25, 0.5, 1.0\}$. The PINN tracks the analytical curve closely at all maturities, including the shallow-slope region deep out-of-the-money and the near-linear in-the-money region.

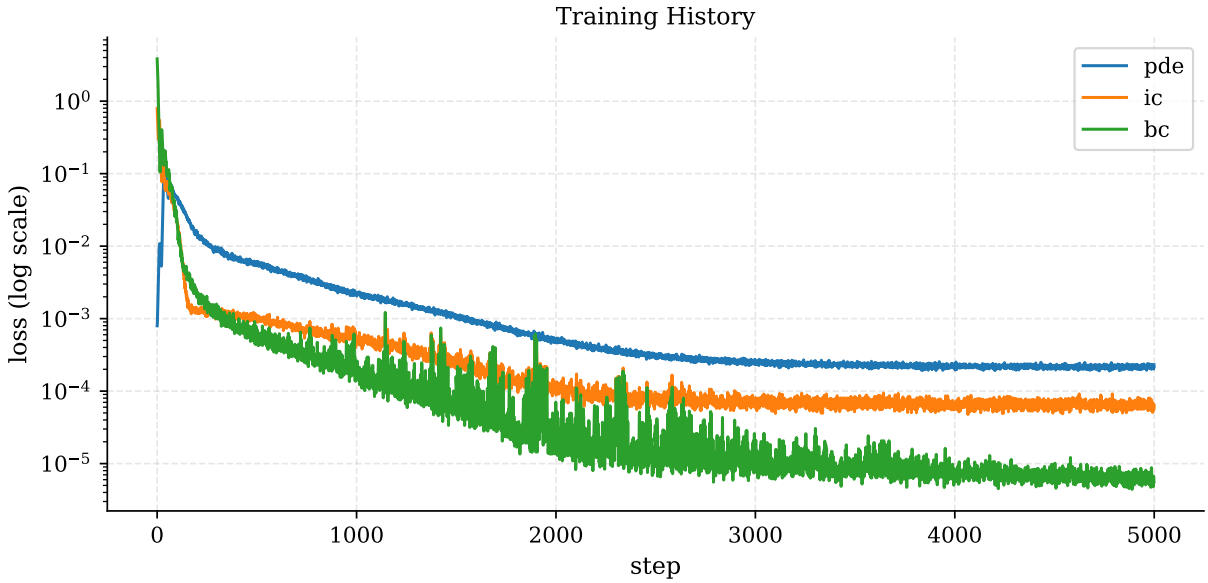

Figure 4. Training loss curves for the Black–Scholes PINN over 5 000 Adam steps, showing the total loss together with its three components: PDE residual $\mathcal{L}_{\text{pde}}$, boundary condition $\mathcal{L}_{\text{bc}}$, and initial condition $\mathcal{L}_{\text{ic}}$ (weighted by $\lambda_{\text{pde}} = 1$, $\lambda_{\text{bc}} = 5$, $\lambda_{\text{ic}} = 10$). The initial-condition term dominates early training before all three components converge.

A.2 Greeks Analysis

Option Greeks are computed by differentiating the trained PINN output with respect to its inputs via automatic differentiation, providing a byproduct of the same computational graph used for the PDE residual. First-order Greeks (Δ, Θ) require one backward pass; second-order Greeks (Γ) require a second pass.

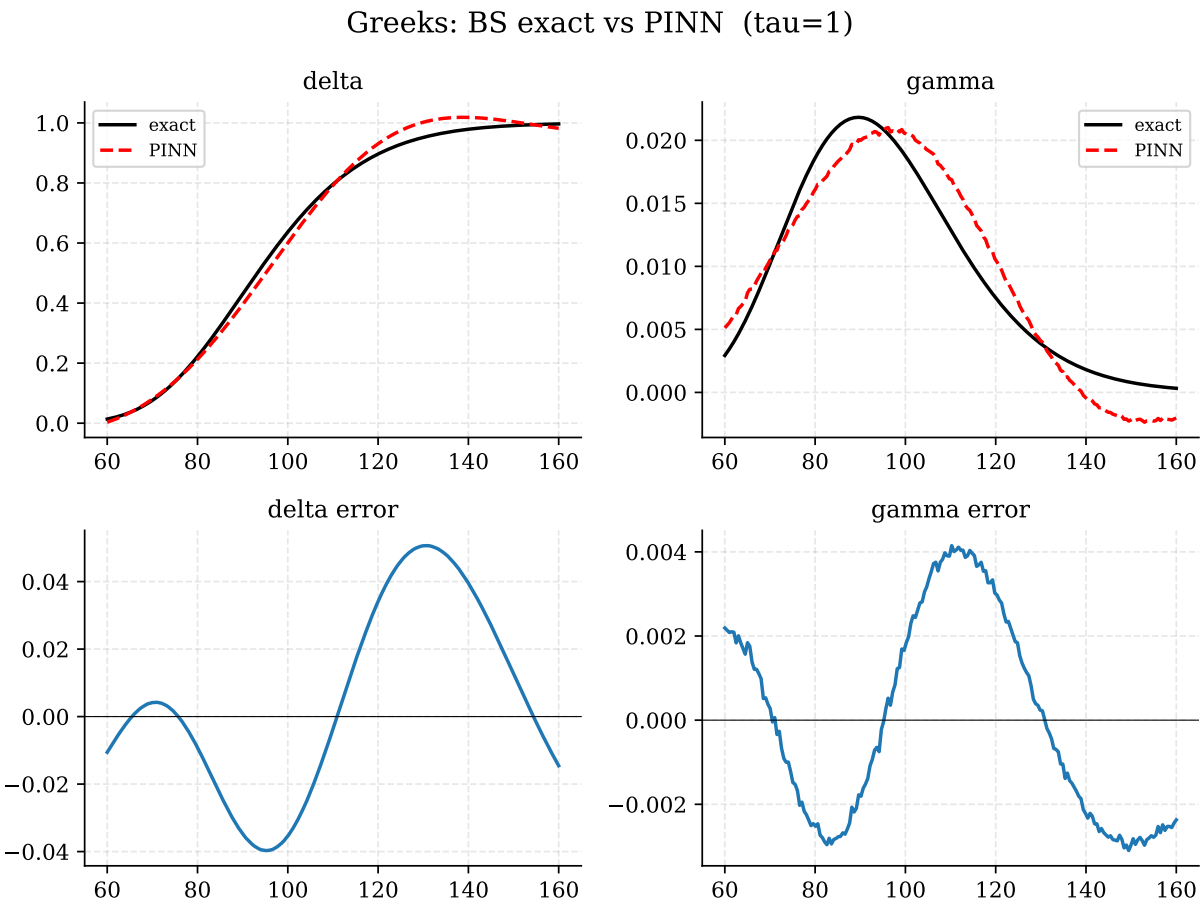

Figure 5. Selected option Greeks (Δ, Γ, Vega, Θ) estimated from the trained Black–Scholes PINN via automatic differentiation, plotted as functions of spot price S at maturity $\tau = 0.5$. The PINN-derived Greeks exhibit the expected qualitative shapes and smoothness, with no finite-difference approximations involved.

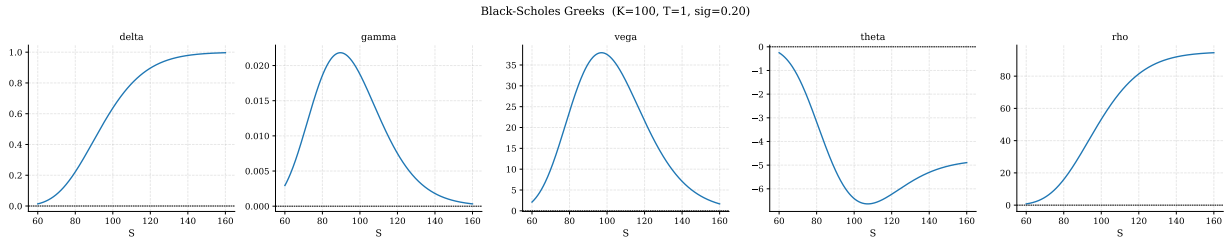

Figure 6. Full comparison of all Greeks between PINN-estimated values and the analytical Black–Scholes formulas across the moneyness range. Higher-order Greeks such as Γ and Vanna show slightly larger relative errors than first-order Greeks, consistent with the fact that each additional differentiation amplifies approximation errors.

A.3 Heston PINN Results

The following figures correspond to the Heston PINN trained on the three-dimensional domain $(S/K,\, v,\, \tau)$ with parameters $\kappa = 2.0$, $\theta = 0.04$, $\xi = 0.3$, $\rho = -0.7$, $v_0 = 0.04$, $r = 0.05$, $K = 100$.

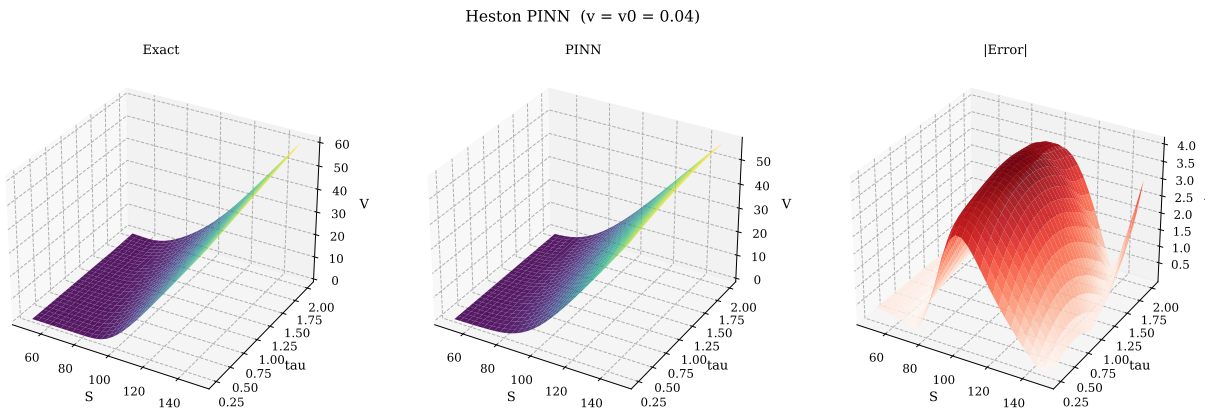

Figure 7. Heston option price surfaces: PINN approximation vs. the semi-analytical benchmark generated by the Fourier-cosine (COS) method (Fang and Oosterlee 2008). Surfaces are shown at a fixed variance $v = \theta = 0.04$ over the $(S/K, \tau)$ plane.

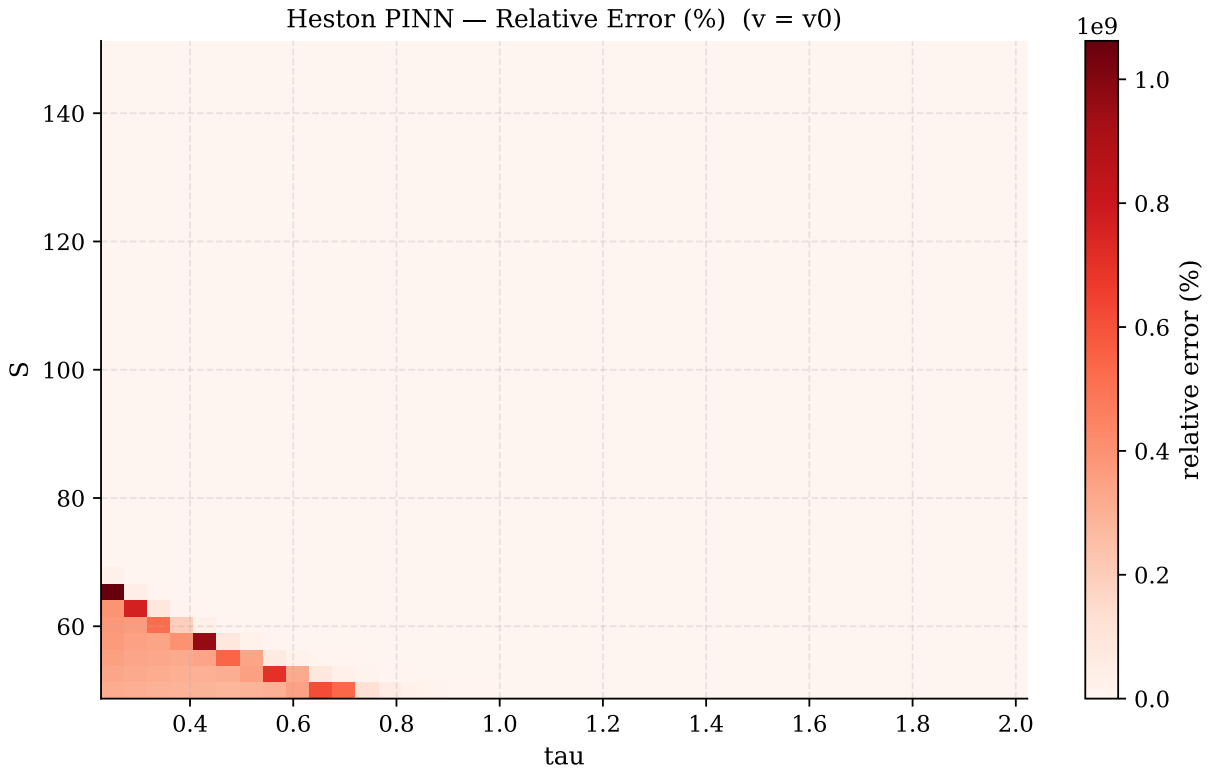

Figure 8. Absolute pointwise error $|\hat{V}_\theta - V_{\text{Heston}}|$ for the Heston PINN across the $(S/K, \tau)$ domain at fixed variance $v = \theta$. The error structure is similar to the Black–Scholes case, with larger deviations near the at-the-money region at short maturities.

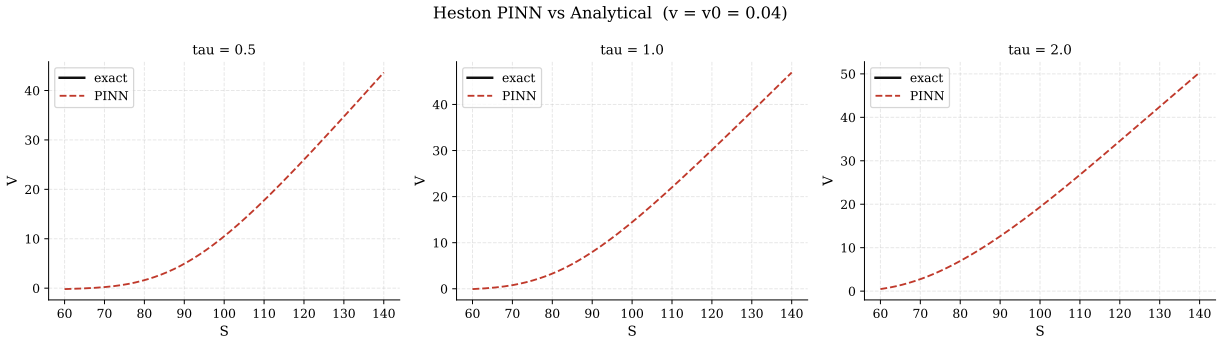

Figure 9. Cross-sectional slices of the Heston PINN solution (solid) and COS benchmark (dashed) at fixed maturities $\tau \in \{0.1, 0.25, 0.5, 1.0\}$. The additional variance dimension of the Heston model is fixed at $v = \theta$.

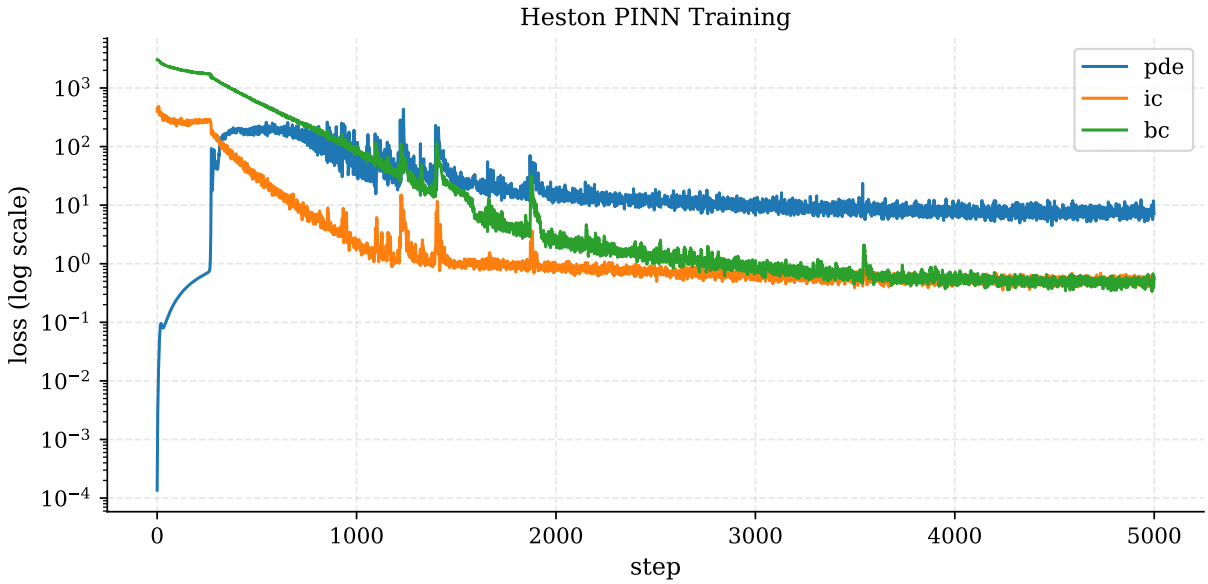

Figure 10. Training loss curves for the Heston PINN over 8 000 Adam steps. The three components — PDE residual, boundary condition, and initial condition — are shown separately. Convergence is slower than the Black–Scholes case, reflecting the higher-dimensional domain and additional mixed-derivative term.

A.4 Calibration Results

The calibration pipeline recovers the five Heston parameters $(\kappa, \theta, \xi, \rho, v_0)$ from a grid of synthetic option prices generated by the COS benchmark. The inverse problem is solved via L-BFGS-B optimization minimizing the mean squared pricing error.

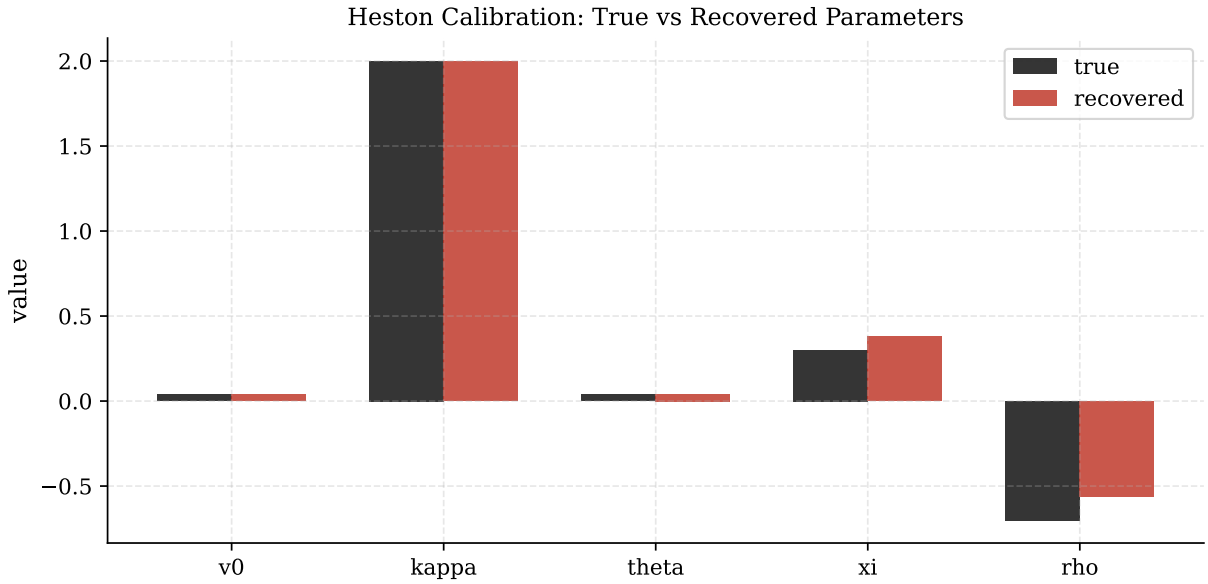

Figure 11. Recovered vs. true Heston parameter values across calibration experiments. Each point represents one calibration run with a different initial guess; the diagonal line indicates perfect recovery. The clustering near the diagonal confirms that the optimizer consistently finds accurate parameter estimates.

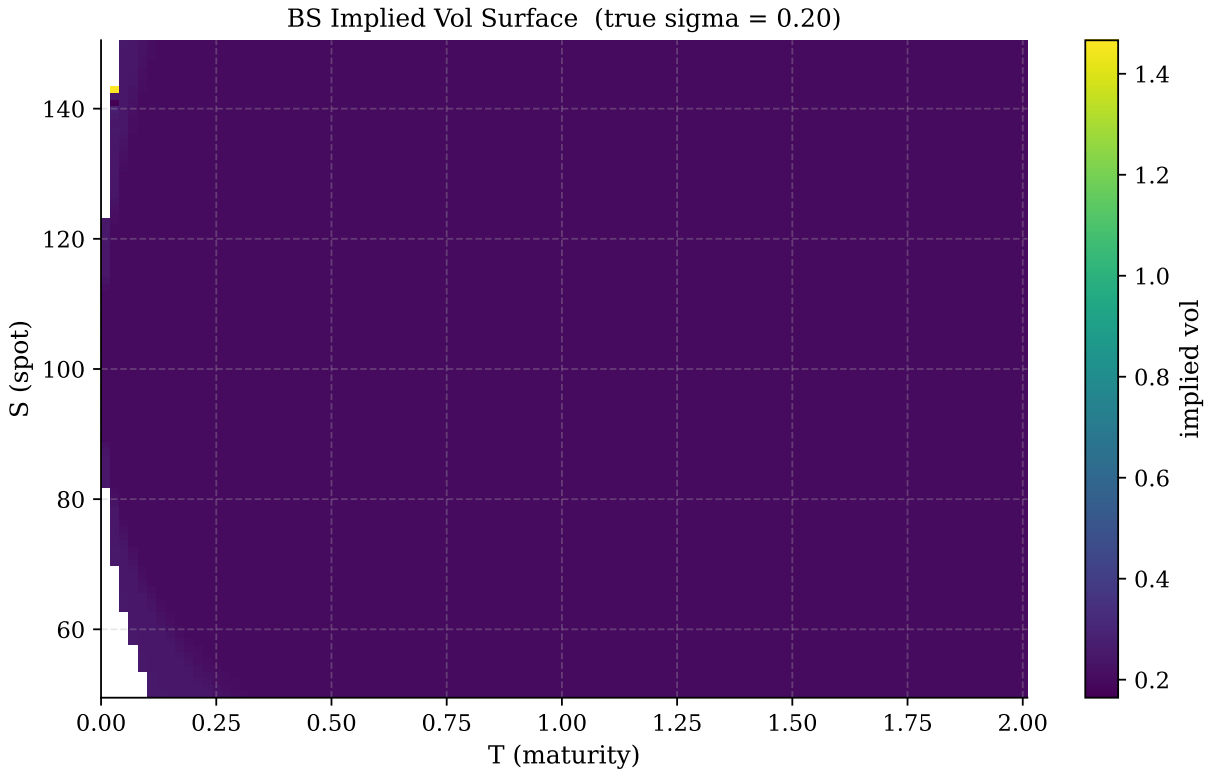

Figure 12. Implied volatility surface heatmap for the calibrated Heston model. The calibrated parameters reproduce the characteristic volatility smile and term structure of the benchmark prices, with darker regions indicating higher implied volatility.